STQA Sheet

1. **Statistical quality control (SQC) methods use seven basic quality management tools, List them.**
2. **What is the difference between state-oriented test-case and state-less-systems test-case, justify your answer with examples.**
3. **Software quality is measured in terms of quality factors and criteria, clarify with an example.**
4. **Distinguish between the following concepts (Failure, error, fault, and defect)**
5. **Distinguish between concepts of *Verification* and *Validation***
6. **Define the concept of "test oracle".**
7. **Write short notes on "Lean principle".**
8. **Write short notes on "Shewhart's PDCA cycle"**
9. **According to Ishikawa, dispersion in product quality came from 4 common causes (4M), list them**
10. **List Key Elements of Total Quality Control (TQC):**
11. **Five Views of Software Quality:**
12. **Write short notes on stages of testing in the V-model**
13. **Compare between acceptance testing and regression testing.**
14. **Compare between White-box and Black-box Testing**
15. **Two practical methods for evaluating test adequacy**
16. **Suppose that T represents a set of test-cases and C is a set of selection criteria, define the following concepts**
    a. **Complete (T,C).**
    b. **Reliable(C).**
    c. **Valid(C).**
17. **List 3 kinds of program faults**
18. **Using CPPChecker, write code analysis (expected errors) for the following segment of code.**

```
a. void f(int i){
b. char *p; *p = 0;
c. for(int j=0; j<i;j++)
d. cout<<list[i];
e. }
```

19. **Two practical methods for evaluating test adequacy**
20. **Write short notes on "Debugging" and its approaches**
21. **List main steps for performing static unit testing**
22. **List main steps to prevent common defects.**
23. **Write short notes on mutation testing**

24. Mutation test

Consider the following program unit p	**Consider the following test suite:**

1. double get_avg_list(int list[], int ms, int sv){
2. double avg=0;
3. int i=0;
4. while (i<ms && list[i]!=sv){
4.1. avg+=list[i];
4.2. i++;
5. }
6. if(i>0)
6.1. avg=avg/i;
7. return avg;
8. }
	Test case 01:
	Input ({1,2,3,4,5},5,-1)
	Output: avg=3
	Test case 02:
	Input ({1,2,3,4,-1},4,-1)
	Output: avg=2.5
	Test case 03:
	Input ({1,2,3,-1},3,-1)
	Output: avg=2

Consider the following mutants: 1. Line 3 i=1 2. Line 4 i<=ms 3. Line 4 list[i]==sv 4. Line 4.2 i=i+1 5. Line 6.1 avg=avg/ms	

Identify types of list mutants

Check if mutation score =100%, if no, discuss what modifications are recommended.

25. Draw a flowchart for test-first process in extreme programming (XP)

26. Define JUnit

27. List unit testing steps

28. (a) List three formulas to compute McCabe's complexity measure, (b) why such measures are recommended before testing, (c) do you think that there are extra measures that software tester has to put in his consideration, if yes, list few of them, (d) using any of McCabe formulas, compute the cyclomatic complexity for the following segment of code.

Node	Statement
(1)	`while(x<100){`
(2)	`if (a[x] % 2 == 0) {`
(3)	`parity = 0;`
	`}`
	`else {`
(4)	`parity = 1;`
(5)	`}`
(6)	`switch(parity){`
	`case 0:`
(7)	`println( "a[" + i + "] is even");`
	`case 1:`
(8)	`println( "a[" + i + "] is odd");`
	`default:`
(9)	`println( "Unexpected error");`
	`}`
(10)	`x++;`
	`}`
(11)	`p = true;`

29. JUnit(Java)/NUnite(C#): suppose that you are recommended to test a class 'student', this class has function called 'assessGPA()', this function uses private attributes "a, b, and c" and returns a float value from 0 to 4, 0.0001 is the recommended tolerance level, Write

student
double a,b,c;
student(double A, double B, double C) assessGPA()

simple test suite code that extends from TestCase in JUnit.framework/ TestTools.UnitTesting to test this function. [Hint: pass private attributes via class constructor]

```
//---------------------------------------- [JUnit – Java] ----------------------------------------
import student;
import junit.framework.*;
public class MyTestSuite ……..……
{
        public void testmethod_01(){ ……
        ……
        ……
        }
        public void testmethod_02(){ ……
        ……
        ……
        }
}
//-----------------------------------------[NUnit – C#] ----------------------------------------
using sudentApp;
using Microsoft.VisualStudio.TestTools.UnitTesting;
namespace MyTestSuite
{
        […….……]
        public class banktest
        {
                […….……]
                public void testmethod_01(){ ……
                ……....
                ……....
                }
                […….……]
                public void testmethod_02(){ ……
                ……....
                ……....
                }
        }
}
```